TikuKits ML: Logistic Regression for Embedded Systems

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 14, 2026

Outline

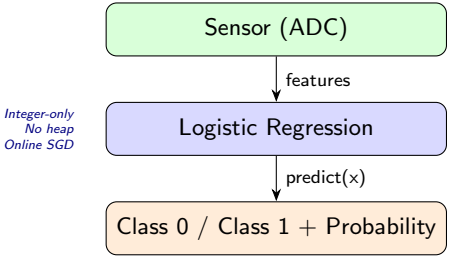
Why Logistic Regression on MCUs?

Embedded Use Cases

- **Threshold detection** — is temperature above safe limit?
- **Fault classification** — vibration exceeds normal range?
- **Anomaly detection** — sensor reading suspicious?
- **Probability output** — confidence-aware decisions

Constraints

- No FPU (16-bit MSP430)
- No heap allocation
- Limited RAM (2–8 KB)
- Every µJ counts



Design Goal

Bring binary logistic regression to devices with **no floating point**, with online SGD training and probability outputs.

Logistic Regression Model

Decision function:

$$f(\mathbf{x}) = w_0 + \sum_{i=1}^{n} w_i \cdot x_i$$

Probability:

$$P(y = 1 \mid \mathbf{x}) = \sigma(f(\mathbf{x}))$$

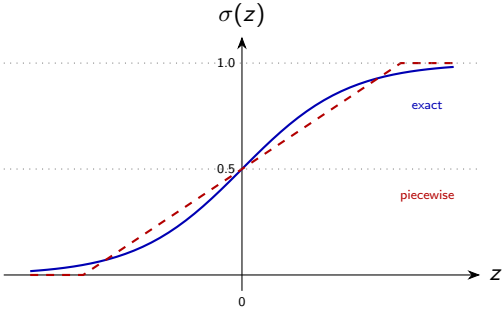
where $\sigma(z) = \frac{1}{1+e^{-z}}$.

Classification:

$$\hat{y} = \begin{cases} 1 & \text{if } \sigma(f(\mathbf{x})) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

SGD update for sample $(\mathbf{x}, y)$:

$$w_j \leftarrow w_j + \eta \cdot (y - \hat{p}) \cdot x_j$$



Integer Sigmoid Approximation

3-piece piecewise linear in Q(shift):

- $z \leq -4$: return 0
- $-4 < z < 4$: return $z/8 + 0.5$

The tiku_kits_ml_logreg Structure

ml/classification/tiku_kits_ml_logreg.h

```
struct tiku_kits_ml_logreg {
    /* bias + feature weights Q(shift) */
    int32_t weights[
        TIKU_KITS_ML_LOGREG_MAX_FEATURES
        + 1];
    uint8_t  n_features;
    uint8_t  shift;
    int32_t  learning_rate;
    uint16_t n;
};
```

Memory footprint:
$(8+1) \times 4 + 1 + 1 + 4 + 2 = 44$ bytes per model.
All statically allocated — no heap.

Weight vector layout:

w[0]	w[1]	w[2]	···
bias	x_1	x_2	

Online Learning

- Train one sample at a time
- No sample buffer needed
- Default learning rate: ~ 0.1 in Q8
- Labels: $y \in \{0, 1\}$

Complete API

Function	Description
`tiku_kits_ml_logreg_init(lr, nf, sh)`	Initialize model with `nf` features, Q-format `sh`
`tiku_kits_ml_logreg_reset(lr)`	Zero all weights, reset training count
`tiku_kits_ml_logreg_set_lr(lr, rate)`	Set learning rate in Q(shift)
`tiku_kits_ml_logreg_train(lr, x, y)`	One SGD step; label $y \in \{0, 1\}$
`tiku_kits_ml_logreg_predict(lr, x, &result)`	Binary prediction: 0 or 1
`tiku_kits_ml_logreg_predict_proba(lr, x, &prob)`	Probability in Q(shift); range $[0, 2^{\text{shift}}]$
`tiku_kits_ml_logreg_get_weights(lr, w)`	Export weight vector (bias + features)
`tiku_kits_ml_logreg_set_weights(lr, w)`	Load pre-trained weights
`tiku_kits_ml_logreg_count(lr)`	Number of training samples seen

Probability Output

`predict_proba()` returns the sigmoid approximation of $f(\mathbf{x})$ in Q(shift). With shift=8, a return value of 128 means 50% confidence.

Compile-Time Configuration

Configurable parameters

```c
/* Maximum input features */
#ifndef TIKU_KITS_ML_LOGREG_MAX_FEATURES
#define TIKU_KITS_ML_LOGREG_MAX_FEATURES 8
#endif

/* Default Q-format shift */
#ifndef TIKU_KITS_ML_LOGREG_SHIFT
#define TIKU_KITS_ML_LOGREG_SHIFT 8
#endif
```

Defaults at init:

- Learning rate: $(1 \ll \text{shift})/10 \approx 0.1$
- All weights zeroed

Runtime configuration

```c
struct tiku_kits_ml_logreg lr;

/* 3 features, 8-bit Q-format */
tiku_kits_ml_logreg_init(&lr, 3, 8);

/* Tune learning rate: ~0.05 */
tiku_kits_ml_logreg_set_lr(&lr, 13);

/* Or load pre-trained weights */
int32_t w[] = {128, 256, -128, 64};
tiku_kits_ml_logreg_set_weights(
    &lr, w);
```

Shift	Scale	Resolution
4	16	0.0625
8	256	0.0039
12	4096	0.00024

Integer Sigmoid and Dot Product

Dot product — int64 accumulation

```c
static int32_t dot_product(
    const struct tiku_kits_ml_logreg *lr,
    const tiku_kits_ml_elem_t *x)
{
    int64_t acc;
    uint8_t i;

    acc = (int64_t)lr->weights[0]; /*bias*/
    for (i = 0; i < lr->n_features; i++) {
        acc += (int64_t)lr->weights[i + 1]
               * (int64_t)x[i];
    }
    return (int32_t)(acc >> lr->shift);
}
```

Piecewise sigmoid approximation

```c
static int32_t sigmoid_q(
    int32_t z, uint8_t shift)
{
    int32_t one_q = (int32_t)(1 << shift);
    int32_t four_q = 4 * one_q;
    int32_t half_q = one_q / 2;

    if (z <= -four_q)
        return 0;
    if (z >= four_q)
        return one_q;

    /* Linear: z/8 + 0.5 */
    return half_q
           + (z * one_q) / (8 * one_q);
}
```

Why Piecewise Linear?

The exact sigmoid requires `exp()` which is unavailable on MSP430. A 3-piece linear approximation captures the S-shape with only integer multiply and divide — sufficient for binary classification on

SGD Training Step

tiku_kits_ml_logreg.c — train

```c
int tiku_kits_ml_logreg_train(struct tiku_kits_ml_logreg *lr,
                              const tiku_kits_ml_elem_t *x, uint8_t y)
{
    int32_t z, p_q, error;
    int64_t lr_err;
    uint8_t i;

    /* Forward: dot product then sigmoid */
    z   = dot_product(lr, x);
    p_q = sigmoid_q(z, lr->shift);

    /* Error: target - predicted (in Q(shift)) */
    error = ((int32_t)(y ? (1 << lr->shift) : 0)) - p_q;

    /* Gradient descent update */
    lr_err = ((int64_t)lr->learning_rate * (int64_t)error) >> lr->shift;

    lr->weights[0] += (int32_t)lr_err;                    /* bias */
    for (i = 0; i < lr->n_features; i++) {
        lr->weights[i + 1] += (int32_t)(lr_err * (int64_t)x[i]);
    }

    lr->n++;
    return TIKU_KITS_ML_OK;
}
```

Example 1: Temperature Alarm

examples/example_ml_logreg.c — classify hot vs. safe

```c
struct tiku_kits_ml_logreg lr;
uint8_t result;

tiku_kits_ml_logreg_init(&lr, 1, 8);   /* 1 feature, Q8 */

/* Train: temperature (deci-C) -> safe(0) / hot(1) */
static const int32_t temps[]  = {200, 220, 240, 300, 320, 340};
static const uint8_t labels[] = {  0,   0,   0,   1,   1,   1};

uint8_t epoch, s;
for (epoch = 0; epoch < 30; epoch++) {
    for (s = 0; s < 6; s++) {
        tiku_kits_ml_logreg_train(&lr, &temps[s], labels[s]);
    }
}

/* Predict on new reading */
int32_t new_temp = 280;
tiku_kits_ml_logreg_predict(&lr, &new_temp, &result);
/* result: 0 or 1 depending on learned boundary */
```

Example 2: Pre-Trained Vibration Classifier

Deploy with pre-computed weights

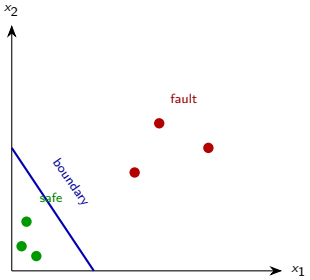
```
struct tiku_kits_ml_logreg lr;
uint8_t result;
int32_t prob;

tiku_kits_ml_logreg_init(&lr, 2, 8);

/* Load pre-trained weights:
 * bias = -5.0, w1 = +3.0, w2 = +2.0
 * in Q8: -1280, +768, +512 */
int32_t w[] = {-1280, 768, 512};
tiku_kits_ml_logreg_set_weights(&lr, w);

/* Classify vibration sample */
tiku_kits_ml_elem_t x[] = {3, 2};
tiku_kits_ml_logreg_predict(&lr, x,
                            &result);
tiku_kits_ml_logreg_predict_proba(&lr, x,
                                  &prob);

/* prob gives confidence in Q8
 * 256 = 100%, 128 = 50% */
```



Pre-Trained Deployment

Train offline (PC/cloud), export weights, deploy on MCU with `set_weights()`. Zero training cost on-device.

Test Cases Overview

#	Test Function	What It Verifies
1	`test_kits_ml_logreg_init`	Proper initialization, parameter validation
2	`test_kits_ml_logreg_pretrained`	Pre-loaded weights produce correct predictions
3	`test_kits_ml_logreg_sigmoid_saturation`	Extreme inputs saturate to 0 or 1
4	`test_kits_ml_logreg_sigmoid_midpoint`	Zero input gives $\sim$0.5 probability
5	`test_kits_ml_logreg_training`	SGD convergence on linearly separable data
6	`test_kits_ml_logreg_two_features`	Multi-feature classification
7	`test_kits_ml_logreg_reset`	Reset clears weights and count
8	`test_kits_ml_logreg_learning_rate`	Custom learning rate accepted
9	`test_kits_ml_logreg_null_inputs`	All functions reject NULL with ERR_NULL

Summary

What We Built

- Binary logistic regression classifier
- Integer-only piecewise sigmoid
- Online SGD training (one sample at a time)
- 44 bytes of state, no heap
- Probability output in Q(shift)

Key Design Choices

- 3-piece linear sigmoid avoids `exp()`
- `int64_t` dot product prevents overflow
- Separate `predict()` and `predict_proba()`
- Pre-trained weight loading supported

Use Cases

- Temperature threshold alarms
- Vibration fault detection
- Motion/gesture binary classification
- Confidence-aware embedded decisions

Files

Header:	`ml/classification/tiku_kits_ml_logreg.h`
Source:	`ml/classification/tiku_kits_ml_logreg.c`
Tests:	`tests/ml/test_kits_ml_logreg.c`
Example:	`examples/example_ml_logreg.c`

Thank You

TikuKits ML: Logistic Regression

Integer-Only Machine Learning for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
Common:	`tikukits/ml/tiku_kits_ml.h`
Header:	`tikukits/ml/classification/tiku_kits_ml_logreg.h`
Source:	`tikukits/ml/classification/tiku_kits_ml_logreg.c`
Tests:	`tikukits/tests/ml/test_kits_ml_logreg.c`
Examples:	`tikukits/examples/example_ml_logreg.c`